Predict My Glucose

From a validated personalized predictor to an evidence-grounded AWS health AI



Core design principle

PMRF owns the numerical prediction and uncertainty. AWS operationalizes the model, retrieves evidence, explains the result, and applies safety controls.

PR #2 changes the discussion: the core service is already deployable

Feature/model now includes a tested FastAPI service, model store, S3 integration, AWS deployment guide, and acceptance tests.

Model + service now implemented

- CPU-only FastAPI container
- Model v4-20261010
- MobileCLIP + LightGBM + transformer ensemble
- /history builds first-week personalization state in ~2–3 s
- No per-user LightGBM retraining

Validation results

- 44 users / 402 day-8+ meals
- Peak-rise MAE: 19.8 mg/dL
- 176 / 176 golden predictions matched report
- 13 / 13 end-to-end API checks passed
- S3 storage / model-download tests passed

Runtime profile

- 2 vCPU / 4 GB target task
- ~1.0 GB memory
- 252 ms median, 374 ms p99 (1 client)
- ~3.8 predictions/s
- ~3 s container startup

What this means for the mentor meeting

We no longer need to ask “How should we host the model?” from scratch. The useful question is whether the current ECS/Fargate pattern is the right hackathon/production trade-off — and where Bedrock, RAG, and Guardrails add the most value next.

Strongest evidence story: personalized prediction is already validated; the next differentiator is explainability + grounded interpretation.

Current AWS deployment architecture from PR #2

This is no longer hypothetical — the deployment guide specifies the service path, storage, auth, scaling, and model versioning.



Amazon S3

Meal-photo uploads via presigned PUTs

Model artifacts: v4-20261010, 466 MB, 115 files

Current task configuration

Encrypted, private, versioned

Amazon EFS

Per-person CGM / HR / meals / meds + first-week state

~1–3 MB per person

Encryption at rest

CloudWatch

JSON request logs

method · route · status · latency

No glucose values or photos logged

Model release

Offline retrain → S3 model version → rolling ECS deploy

Checksums verified at startup

ECS Fargate: Linux x86_64, 2 vCPU, 4 GB, port 8080, one uvicorn worker.

API Gateway HTTP API routes /v1/* and /healthz; first-week upload is ~1–2 MB.

Autoscaling targets CPU 60%; one task serves ~3.5–3.8 predictions/s.

What the live API already exposes for explainability

The model output is richer than a glucose curve — it already provides uncertainty, contributions, similar meals, what-ifs, and metabolic profile fields.

Prediction

Curve + G30/60/90/120

Peak / rise / time-to-peak / iAUC

Calibrated 80% prediction band

Why this prediction

Baseline + factor contributions

Meal / CGM / time / HR / meds

Similar meals (you + others)

Personalization

History meals + average rise

Carb sensitivity + mean glucose

Resting HR + personal profile

Profile + what-if

HbA1c estimate + range; GRI zone

Status probability*; relative T2D tier

What-if scenarios

*** API contract: normal / prediabetes / diabetes status must never be presented as a diagnosis.**

Important distinction for our new idea

The existing API gives model confidence flags and predictive uncertainty. Our Bedrock layer should add evidence-grounded physiological interpretation and citations — not replace PMRF uncertainty with an LLM "confidence score."

Next layer: medical knowledge + GI/GL for stronger explainability

Keep these as post-prediction interpretation features for the hackathon so we do not invalidate the existing model.



Medical knowledge–grounded interpretation

Retrieve peer-reviewed CGM / nutrition evidence that could plausibly explain high excursions, delayed recovery, or variability. Return citations + evidence-support signal.

Meal-level GI / GL workflow

Recognized foods + estimated portions → available carbs → validated GI table lookup → food-level GL → meal-level weighted GI / total GL.

Scope rule for the hackathon

GI/GL is explanation-only. If we later want it inside PMRF, retrain + revalidate. Medical interpretation is decision support, not diagnosis.

User-facing goal: "What is predicted?" + "Why?" + "What may physiologically explain it?" + "How well is that explanation supported?"

Mentor discussion: decisions that matter now

PR #2 answers many implementation questions; the mentor can now focus on architecture trade-offs and the evidence-grounded AI layer.

1 · Is ECS/Fargate the right endpoint?

We already have a CPU-only FastAPI container at ~252 ms median and ~1 GB memory. Is API Gateway → ALB → Fargate the right level of complexity for the hackathon, or should we simplify?

2 · EFS now vs. DynamoDB later?

Current per-person state is ~1–3 MB on EFS with atomic file writes. At what scale or concurrency would you move signals/state to DynamoDB or another managed store?

3 · Best Bedrock pattern?

What is the simplest way to add curated medical RAG, citations, Guardrails, and a grounding/evidence-support check while minimizing sensitive data sent to the LLM?

4 · Health-data posture

Current service already supports Cognito auth, encrypted S3/EFS, pseudonymous IDs, and CloudWatch logs without glucose/photos. What additional controls are worth showing in a hackathon demo?

5 · Scope priority

If time is tight, should we prioritize medical RAG + citations over GI/GL? Our proposed rule: PMRF → secure AWS path → Bedrock + Guardrails → medical RAG → GI/GL only if time remains.

Suggested mentor narrative: “The prediction engine is working and tested. We want AWS guidance on the simplest production path and the safest, most credible explanation layer.”